

Project Description Document

Project Title: Classical Machine Learning Project: Numerical Regression and Image Recognition

Prepared for: University machine learning project submission

Student / Team: _____

Course: _____

Date: 2026-05-09

1. Project Overview

This project implements two required machine learning tasks using only the datasets already provided inside the project folder.

- Numerical dataset task: Linear Regression and KNN Regressor.
- Image dataset task: Logistic Regression classifier and KMeans clustering-based classifier.

The implementation uses a complete train, validation, and test workflow. It saves trained models, preprocessing objects, evaluation reports, plots, and confusion matrices.

2. General Dataset Information

2.1 Numerical Dataset

Item	Value
Dataset name	Cyber Security Salaries Dataset
Dataset source	User-provided CSV file placed in datasets/numerical/
Dataset file	salaries_cyber.csv
Problem type	Regression
Target column	salary_in_usd
Total samples	1,247
Original shape	[1247, 11]
Used shape	[1247, 9]
Raw input columns	work_year, experience_level, employment_type, job_title, employee_residence, remote_ratio, company_location, company_size
Numerical columns	work_year, remote_ratio
Categorical columns	experience_level, employment_type, job_title, employee_residence, company_location, company_size
Training samples	872
Validation samples	187
Testing samples	188

The numerical dataset is used for regression. Therefore, the correct model outputs are continuous numeric predictions, not class labels.

2.2 Image Dataset

Item	Value
Dataset name	Flowers Recognition Image Dataset
Dataset source	User-provided image folders placed in datasets/images/
Problem type	Image classification / clustering-based labeling
Number of classes	5
Class labels	daisy, dandelion, rose, sunflower, tulip
Total image samples	4,317
Image size used	128 x 128 pixels
Training samples	3,021
Validation samples	648
Testing samples	648

Class sample counts:

- daisy: 764 images

- dandelion: 1,052 images
- rose: 784 images
- sunflower: 733 images
- tulip: 984 images

The image implementation is limited to a maximum of five classes, satisfying the project requirement.

3. Implementation Details

3.1 Numerical Preprocessing

The numerical pipeline uses a scikit-learn preprocessing structure with separate handling for numerical and categorical columns.

- Missing numerical values are filled using median imputation.
- Missing categorical values are filled using most-frequent imputation.
- Numerical outliers are handled conservatively with IQR-based capping.
- Numerical features are scaled using StandardScaler.
- Categorical features are converted using OneHotEncoder.
- The preprocessing is fitted only on training data during model selection to avoid data leakage.

The target column was checked for skewness. The recorded target skewness is 2.4795. A $\log_{1p}$ target transformation was tested, but the final model used it only if validation RMSE improved.

3.2 Numerical Feature Extraction Phase

For the numerical task, feature extraction means transforming the original mixed-type columns into a fully numeric machine learning matrix.

Item	Value
Original raw input features	8
Final extracted/encoded features	206
Training feature dimension	[872, 198]
Validation feature dimension	[187, 198]
Testing feature dimension	[188, 198]

Raw feature names:

```
work_year, experience_level, employment_type, job_title, employee_residence, remote_ratio,
company_location, company_size
```

Encoded feature names generated by the preprocessing pipeline:

```
work_year, remote_ratio, experience_level_EN, experience_level_EX, experience_level_MI,
experience_level_SE, employment_type_CT, employment_type_FL, employment_type_FT,
employment_type_PT, job_title_Application Security Analyst, job_title_Application Security
Architect, job_title_Application Security Engineer, job_title_Application Security
Specialist, job_title_Azure Security Engineer, job_title_Chief Information Security Officer,
job_title_Cloud Security Architect, job_title_Cloud Security Engineer, job_title_Cloud
Security Engineering Manager, job_title_Computer Forensic Software Engineer,
job_title_Concierge Security Engineer, job_title_Corporate Infrastructure Security Engineer,
job_title_Corporate Security Engineer, job_title_Cyber Program Manager, job_title_Cyber
Security Analyst, job_title_Cyber Security Architect, job_title_Cyber Security Consultant,
job_title_Cyber Security Engineer, job_title_Cyber Security Researcher, job_title_Cyber
Security Specialist, job_title_Cyber Security Training Specialist, job_title_Cyber Threat
Analyst, job_title_Cyber Threat Intelligence Analyst, job_title_Data Security Analyst,
job_title_Detection Engineer, job_title_DevOps Security Engineer, job_title_DevSecOps
Engineer, job_title_Digital Forensics Analyst, job_title_Director of Information Security,
job_title_Enterprise Security Engineer, job_title_Ethical Hacker, job_title_Head of
Information Security, job_title_Head of Security, job_title_IAM Engineer, job_title_IT
Security Analyst, job_title_IT Security Engineer, job_title_IT Security Manager,
job_title_Incident Response Analyst, job_title_Incident Response Lead, job_title_Incident
```

Response Manager, job_title_Information Security Analyst, job_title_Information Security Architect, job_title_Information Security Compliance Analyst, job_title_Information Security Compliance Lead, job_title_Information Security Compliance Manager, job_title_Information Security Engineer, job_title_Information Security Manager, job_title_Information Security Officer, job_title_Information Security Specialist, job_title_Information Systems Security Engineer, job_title_Infrastructure Security Engineer, job_title_Lead Application Security Engineer, job_title_Lead Information Security Engineer, job_title_Lead Security Engineer, job_title_Network Security Engineer, job_title_Network and Security Engineer, job_title_Offensive Security Engineer, job_title_Penetration Tester, job_title_Penetration Testing Engineer, job_title_Principal Cloud Security Engineer, job_title_Principal Security Engineer, job_title_Privacy Manager, job_title_Product Security Engineer, job_title_SOC Analyst, job_title_Security Analyst, job_title_Security Consultant, job_title_Security DevOps Engineer, job_title_Security Engineer, job_title_Security Engineering Manager, job_title_Security Incident Response Engineer, job_title_Security Officer, job_title_Security Officer 3, job_title_Security Operations Analyst, job_title_Security Operations Engineer, job_title_Security Researcher, job_title_Security Specialist, job_title_Software Security Engineer, job_title_Staff Application Security Engineer, job_title_Staff Security Engineer, job_title_Threat Hunting Lead, job_title_Threat Intelligence Analyst, job_title_Threat Intelligence Response Analyst, job_title_Vulnerability Analyst, job_title_Vulnerability Management Engineer, job_title_Vulnerability Researcher, employee_residence_AE, employee_residence_AF, employee_residence_AR, employee_residence_AT, employee_residence_AU, employee_residence_AZ, employee_residence_BE, employee_residence_BG, employee_residence_BR, employee_residence_BW, employee_residence_CA, employee_residence_CH, employee_residence_CL, employee_residence_CZ, employee_residence_DE, employee_residence_DK, employee_residence_DZ, employee_residence_EE, employee_residence_EG, employee_residence_ES, employee_residence_ET, employee_residence_FR, employee_residence_GB, employee_residence_GH, employee_residence_GR, employee_residence_HU, employee_residence_ID, employee_residence_IE, employee_residence_IL, employee_residence_IN, employee_residence_IR, employee_residence_IT, employee_residence_JP, employee_residence_KE, employee_residence_LT, employee_residence_LU, employee_residence_MX, employee_residence_NG, employee_residence_NL, employee_residence_NO, employee_residence_NZ, employee_residence_PK, employee_residence_PL, employee_residence_PT, employee_residence_RO, employee_residence_RU, employee_residence_SA, employee_residence_SE, employee_residence_SG, employee_residence_SI, employee_residence_TR, employee_residence_TW, employee_residence_US, employee_residence_VN, employee_residence_ZA, company_location_AE, company_location_AQ, company_location_AR, company_location_AT, company_location_AU, company_location_AX, company_location_AZ, company_location_BE, company_location_BR, company_location_BW, company_location_CA, company_location_CH, company_location_CL, company_location_CZ, company_location_DE, company_location_DK, company_location_DZ, company_location_EE, company_location_EG, company_location_ES, company_location_ET, company_location_FR, company_location_GB, company_location_GR, company_location_HU, company_location_ID, company_location_IE, company_location_IL, company_location_IN, company_location_IT, company_location_JP, company_location_KE, company_location_LU, company_location_MX, company_location_NL, company_location_NO, company_location_NZ, company_location_PK, company_location_PL, company_location_PT, company_location_RO, company_location_RS, company_location_RU, company_location_SA, company_location_SE, company_location_SG, company_location_SI, company_location_TR, company_location_TW, company_location_UM, company_location_US, company_location_VN, company_location_ZA, company_size_L, company_size_M, company_size_S

3.3 Image Preprocessing

The image pipeline reads images from class folders, resizes each image to a fixed size, normalizes pixel values, and extracts a fixed-length feature vector.

- Folder format: datasets/images/class_name/image_file
- Supported classes in this run: daisy, dandelion, rose, sunflower, tulip
- Resized image size: 128 x 128
- Feature values are scaled before training Logistic Regression and KMeans.
- Validation and test images are not augmented, protecting fair evaluation.

3.4 Image Feature Extraction Phase

The image feature extractor converts each image into a numeric vector. This is required because Logistic Regression and KMeans cannot directly train on raw image files.

Item	Value
Number of extracted image features	769
Training feature dimension	[3021, 769]
Validation feature dimension	[648, 769]

Item	Value
Testing feature dimension	[648, 769]

Feature types used:

- RGB color histogram features.
- Grayscale histogram features.
- RGB channel statistics.
- Grayscale and gradient statistics.
- Edge-density feature.
- Local grid color statistics.
- HOG-style gradient orientation features.

Image feature names:

```
red_hist_bin_0, red_hist_bin_1, red_hist_bin_2, red_hist_bin_3, red_hist_bin_4,
red_hist_bin_5, red_hist_bin_6, red_hist_bin_7, red_hist_bin_8, red_hist_bin_9,
red_hist_bin_10, red_hist_bin_11, red_hist_bin_12, red_hist_bin_13, red_hist_bin_14,
red_hist_bin_15, green_hist_bin_0, green_hist_bin_1, green_hist_bin_2, green_hist_bin_3,
green_hist_bin_4, green_hist_bin_5, green_hist_bin_6, green_hist_bin_7, green_hist_bin_8,
green_hist_bin_9, green_hist_bin_10, green_hist_bin_11, green_hist_bin_12,
green_hist_bin_13, green_hist_bin_14, green_hist_bin_15, blue_hist_bin_0, blue_hist_bin_1,
blue_hist_bin_2, blue_hist_bin_3, blue_hist_bin_4, blue_hist_bin_5, blue_hist_bin_6,
blue_hist_bin_7, blue_hist_bin_8, blue_hist_bin_9, blue_hist_bin_10, blue_hist_bin_11,
blue_hist_bin_12, blue_hist_bin_13, blue_hist_bin_14, blue_hist_bin_15,
grayscale_hist_bin_0, grayscale_hist_bin_1, grayscale_hist_bin_2, grayscale_hist_bin_3,
grayscale_hist_bin_4, grayscale_hist_bin_5, grayscale_hist_bin_6, grayscale_hist_bin_7,
grayscale_hist_bin_8, grayscale_hist_bin_9, grayscale_hist_bin_10, grayscale_hist_bin_11,
grayscale_hist_bin_12, grayscale_hist_bin_13, grayscale_hist_bin_14, grayscale_hist_bin_15,
grayscale_hist_bin_16, grayscale_hist_bin_17, grayscale_hist_bin_18, grayscale_hist_bin_19,
grayscale_hist_bin_20, grayscale_hist_bin_21, grayscale_hist_bin_22, grayscale_hist_bin_23,
grayscale_hist_bin_24, grayscale_hist_bin_25, grayscale_hist_bin_26, grayscale_hist_bin_27,
grayscale_hist_bin_28, grayscale_hist_bin_29, grayscale_hist_bin_30, grayscale_hist_bin_31,
red_mean, red_std, red_min, red_max, green_mean, green_std, green_min, green_max, blue_mean,
blue_std, blue_min, blue_max, grayscale_mean, grayscale_std, gradient_mean, gradient_std,
edge_density, grid_0_0_red_mean, grid_0_0_red_std, grid_0_0_green_mean, grid_0_0_green_std,
grid_0_0_blue_mean, grid_0_0_blue_std, grid_0_1_red_mean, grid_0_1_red_std,
grid_0_1_green_mean, grid_0_1_green_std, grid_0_1_blue_mean, grid_0_1_blue_std,
grid_0_2_red_mean, grid_0_2_red_std, grid_0_2_green_mean, grid_0_2_green_std,
grid_0_2_blue_mean, grid_0_2_blue_std, grid_0_3_red_mean, grid_0_3_red_std,
grid_0_3_green_mean, grid_0_3_green_std, grid_0_3_blue_mean, grid_0_3_blue_std,
grid_1_0_red_mean, grid_1_0_red_std, grid_1_0_green_mean, grid_1_0_green_std,
grid_1_0_blue_mean, grid_1_0_blue_std, grid_1_1_red_mean, grid_1_1_red_std,
grid_1_1_green_mean, grid_1_1_green_std, grid_1_1_blue_mean, grid_1_1_blue_std,
grid_1_2_red_mean, grid_1_2_red_std, grid_1_2_green_mean, grid_1_2_green_std,
grid_1_2_blue_mean, grid_1_2_blue_std, grid_1_3_red_mean, grid_1_3_red_std,
grid_1_3_green_mean, grid_1_3_green_std, grid_1_3_blue_mean, grid_1_3_blue_std,
grid_2_0_red_mean, grid_2_0_red_std, grid_2_0_green_mean, grid_2_0_green_std,
grid_2_0_blue_mean, grid_2_0_blue_std, grid_2_1_red_mean, grid_2_1_red_std,
grid_2_1_green_mean, grid_2_1_green_std, grid_2_1_blue_mean, grid_2_1_blue_std,
grid_2_2_red_mean, grid_2_2_red_std, grid_2_2_green_mean, grid_2_2_green_std,
grid_2_2_blue_mean, grid_2_2_blue_std, grid_2_3_red_mean, grid_2_3_red_std,
grid_2_3_green_mean, grid_2_3_green_std, grid_2_3_blue_mean, grid_2_3_blue_std,
grid_3_0_red_mean, grid_3_0_red_std, grid_3_0_green_mean, grid_3_0_green_std,
grid_3_0_blue_mean, grid_3_0_blue_std, grid_3_1_red_mean, grid_3_1_red_std,
grid_3_1_green_mean, grid_3_1_green_std, grid_3_1_blue_mean, grid_3_1_blue_std,
grid_3_2_red_mean, grid_3_2_red_std, grid_3_2_green_mean, grid_3_2_green_std,
grid_3_2_blue_mean, grid_3_2_blue_std, grid_3_3_red_mean, grid_3_3_red_std,
grid_3_3_green_mean, grid_3_3_green_std, grid_3_3_blue_mean, grid_3_3_blue_std,
hog_cell_0_0_bin_0, hog_cell_0_0_bin_1, hog_cell_0_0_bin_2, hog_cell_0_0_bin_3,
hog_cell_0_0_bin_4, hog_cell_0_0_bin_5, hog_cell_0_0_bin_6, hog_cell_0_0_bin_7,
hog_cell_0_0_bin_8, hog_cell_0_1_bin_0, hog_cell_0_1_bin_1, hog_cell_0_1_bin_2,
hog_cell_0_1_bin_3, hog_cell_0_1_bin_4, hog_cell_0_1_bin_5, hog_cell_0_1_bin_6,
hog_cell_0_1_bin_7, hog_cell_0_1_bin_8, hog_cell_0_2_bin_0, hog_cell_0_2_bin_1,
hog_cell_0_2_bin_2, hog_cell_0_2_bin_3, hog_cell_0_2_bin_4, hog_cell_0_2_bin_5,
hog_cell_0_2_bin_6, hog_cell_0_2_bin_7, hog_cell_0_2_bin_8
```

The remaining 549 feature names continue using the same generated naming pattern inside the saved project feature extractor.

4. Hyperparameters Used

4.1 Linear Regression

Hyperparameter	Value
Model	scikit-learn LinearRegression
Learning rate	Not applicable: closed-form ordinary least squares
Optimizer	Not applicable: scikit-learn LinearRegression uses least-squares optimization
Regularization	None
Batch size	Not applicable
Number of epochs	Not applicable
fit_intercept	True
positive	False
selected_log1p_transform	False

4.2 KNN Regressor

Hyperparameter	Value
Model	scikit-learn KNeighborsRegressor
Learning rate	Not applicable: instance-based method
Optimizer	Not applicable
Regularization	Not applicable
Batch size	Not applicable
Number of epochs	Not applicable
Candidate n_neighbors	[3, 5, 7, 9, 11, 15, 21]
Candidate weights	['uniform', 'distance']
Candidate metrics	['euclidean', 'manhattan']
CV folds	5
Selected n_neighbors	11
Selected weights	uniform
Selected metric	manhattan

4.3 Logistic Regression

Hyperparameter	Value
Model	scikit-learn LogisticRegression
Learning rate	Controlled internally by lbfgs solver
Optimizer	lbfgs
Regularization	L2 regularization with tuned C value
Batch size	Not applicable: full-batch deterministic solver

Hyperparameter	Value
Number of epochs	Not applicable: max_iter controls solver iterations
Penalty	l2
Solver	lbfgs
Candidate C values	[0.01, 0.1, 1.0, 10.0]
Candidate max_iter	[1000, 2000, 3000]
Candidate class_weight	[None, 'balanced']
Candidate PCA components	[None, 120, 180]
Selected C	0.1
Selected max_iter	1000
Selected class_weight	balanced
Selected PCA components	180

4.4 KMeans

Hyperparameter	Value
Model	scikit-learn KMeans
Learning rate	Not applicable: centroid update algorithm
Optimizer	KMeans iterative centroid minimization
Regularization	None
Batch size	Not applicable
Number of epochs	Not applicable: max_iter=300 controls iterations
n_clusters	5
init	k-means++
selected n_init	30
max_iter	400
algorithm	lloyd
selected PCA components	None

KMeans is unsupervised, so it does not naturally output class labels. The implementation maps clusters to labels by majority voting on the training data.

Cluster-to-label mapping:

- Cluster 0: daisy
- Cluster 1: sunflower
- Cluster 2: dandelion
- Cluster 3: tulip
- Cluster 4: dandelion

5. Results Details on Testing Data

Important note: accuracy and confusion matrix are classification metrics. They are meaningful for Logistic Regression and KMeans in the image task. For Linear Regression and KNN Regressor, the correct testing metrics are MAE, MSE, RMSE, and R2 because the output is numeric and continuous.

5.1 Regression Results

Model	MAE	MSE	RMSE	R2	Accuracy	Confusion Matrix
Linear Regression	32,392.65	1,975,070,209.27	44,441.76	0.4346	Not applicable	Not applicable
KNN Regressor	35,912.42	2,241,113,673.31	47,340.40	0.3584	Not applicable	Not applicable

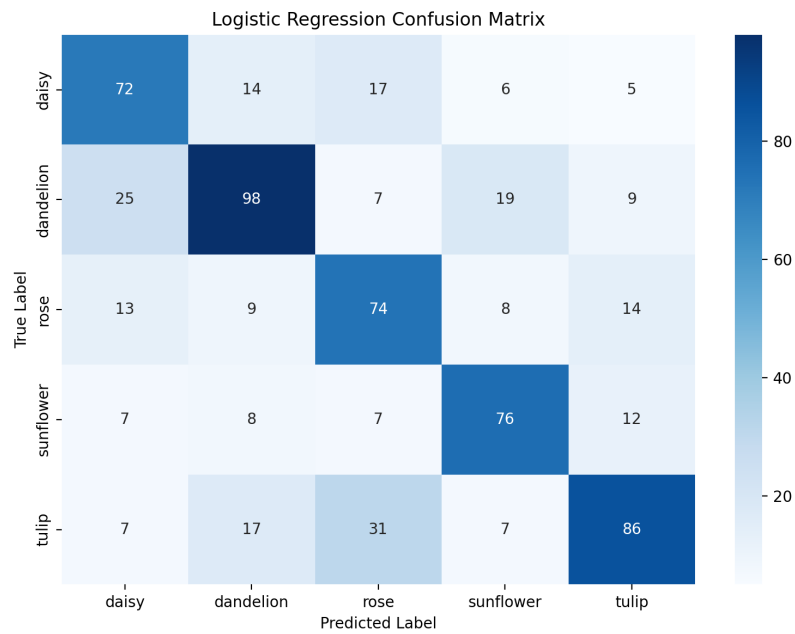
Linear Regression achieved the better regression result in this run because it had lower RMSE and higher R2 on the test set.

5.2 Logistic Regression Image Classification Results

Metric	Value
Test accuracy	62.65%
Testing samples	648
Classes	daisy, dandelion, rose, sunflower, tulip

Confusion matrix for Logistic Regression on testing data:

True / Predicted	daisy	dandelion	rose	sunflower	tulip
daisy	72	14	17	6	5
dandelion	25	98	7	19	9
rose	13	9	74	8	14
sunflower	7	8	7	76	12
tulip	7	17	31	7	86



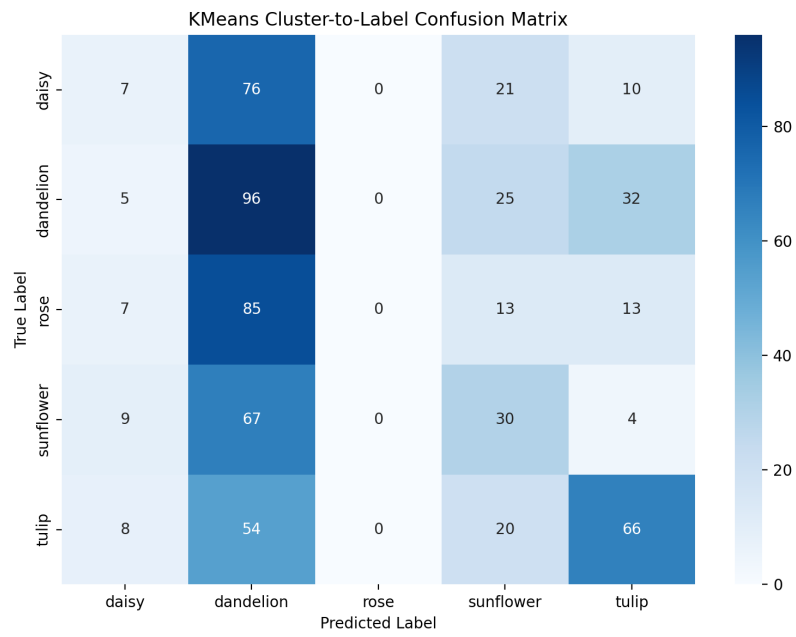
Logistic Regression confusion matrix on testing data.

5.3 KMeans Image Classification Results

Metric	Value
Test accuracy	30.71%
Testing samples	648
Classes	daisy, dandelion, rose, sunflower, tulip

Confusion matrix for KMeans on testing data:

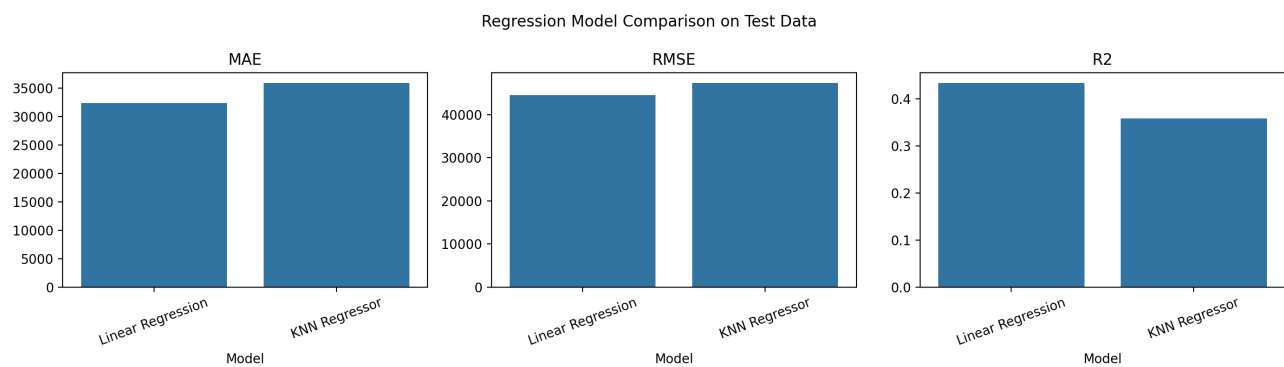
True / Predicted	daisy	dandelion	rose	sunflower	tulip
daisy	7	76	0	21	10
dandelion	5	96	0	25	32
rose	7	85	0	13	13
sunflower	9	67	0	30	4
tulip	8	54	0	20	66



KMeans confusion matrix after cluster-to-label mapping on testing data.

6. Final Model Comparison

Task	Model	Best use	Test result
Numerical regression	Linear Regression	Continuous target prediction	RMSE 44,441.76, R2 0.4346
Numerical regression	KNN Regressor	Distance-based continuous prediction	RMSE 47,340.40, R2 0.3584
Image classification	Logistic Regression	Supervised image classification	Accuracy 62.65%
Image clustering/classification	KMeans	Unsupervised clustering with label mapping	Accuracy 30.71%



Regression model comparison chart.

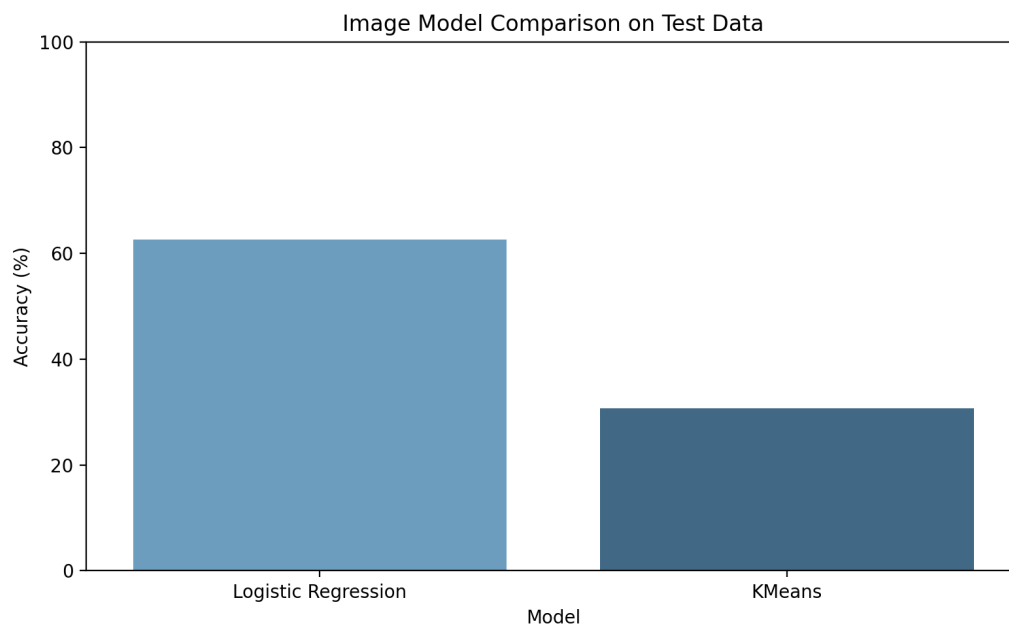


Image model comparison chart.

7. Conclusion

This Project Description Document summarizes the required dataset details, implementation details, hyperparameters, extracted feature dimensions, and testing results.

The numerical task was evaluated using regression metrics because it predicts a continuous target. The image task was evaluated using classification metrics, including accuracy and confusion matrices.

The results show that Linear Regression performed better than KNN Regressor on the numerical test set, while Logistic Regression performed better than KMeans on the image test set. This is expected because Logistic Regression is supervised, whereas KMeans is unsupervised and must be adapted to classification using cluster-to-label majority voting.